



Segundo Trabajo Práctico

CURSO:
INTELIGENCIA ARTIFICIAL

CLASIFICACIÓN DE TEXTOS SEGÚN SU COMPLEJIDAD

Profesor:
Ph. D. Saúl Calderón Ramírez

Estudiantes:
Alejandro Cerdas
Kener Castillo
Pablo Pérez

18 de mayo de 2026

Índice

1. Introducción	2
2. Metodología general	2
3. Perceptrón	3
4. Regresión logística	4
4.1. Diferencias respecto al perceptrón	5
4.2. Condición de Parada Temprana (Early Stopping)	6
5. Resultados con datos sintéticos	7
5.1. Datos linealmente separables	7
5.2. Datos no linealmente separables	8
5.3. Resultados del entrenamiento	9
5.4. Resultados promedio para 10 corridas	10
6. Regresión logística con TF-IDF	11
6.1. Preprocesamiento	11
6.1.1. Análisis de la lematización y la extracción de “stopwords”	13
6.2. Construcción de descriptores	14
6.2.1. Análisis de la transformación TF-IDF	14
6.3. Clasificación y resultados	15
7. Regresión logística con vectores empotrados	16
7.1. Proyección de textos	17
7.2. Clasificación y resultados	19
8. Modelos de lenguaje grandes (LLMs)	19
8.1. SmolLM3	20
8.2. Phi-4 mini	20
8.3. Definición del Prompt	21
8.4. Clasificación sin few-shot	22
8.4.1. Ejecución de ejemplo	22
8.4.2. Ejecución con el dataset completo sin few shots	23
8.5. Clasificación con few-shot	23
8.5.1. Ejecución de ejemplo	24
8.5.2. Ejecución con el dataset completo	25
9. Prueba completa	25
10. Prueba estadística	28
11. Conclusiones	32

1. Introducción

La simplificación de textos estudia métodos para transformar o seleccionar expresiones más comprensibles sin destruir su contenido semántico. En contextos de educación financiera, esta tarea resulta especialmente importante: los textos técnicos suelen contener términos especializados, estructuras sintácticas extensas y referencias conceptuales que pueden dificultar el acceso a la información. Esta dificultad es aún más relevante para poblaciones con necesidades específicas de accesibilidad, como las personas con discapacidad visual, para quienes la claridad textual puede afectar directamente la autonomía en la lectura y comprensión de materiales educativos [1].

En este trabajo se aborda una tarea relacionada: clasificar segmentos de texto en español según su complejidad. Para ello se comparan cuatro familias de clasificadores: regresión logística con descriptores TF-IDF, regresión logística con vectores empotrados, y dos modelos de lenguaje grandes disponibles en Hugging Face. La clase 0 representa textos simples, mientras que la clase 1 representa textos complejos. El conjunto de datos usado es FEINA, compuesto por pares de segmentos financieros simples y complejos [1].

El reporte se divide en dos bloques. Primero, se implementa y analiza la regresión logística a partir de una implementación base del perceptrón, usando datos sintéticos linealmente separables y no linealmente separables. Segundo, se aplica esa regresión logística a la clasificación de textos mediante representaciones TF-IDF y vectores empotrados, y se comparan sus resultados contra clasificadores basados en LLMs bajo configuraciones sin ejemplos y con few-shot learning.

2. Metodología general

El objetivo del presente documento es realizar una comparación experimental entre modelos de clasificación. Por esta razón, el diseño del trabajo separa tres elementos: representación de los datos, clasificador y protocolo de evaluación. Esta separación permite interpretar si una mejora proviene del algoritmo de clasificación, de la forma en que se representa el texto o de una diferencia en las condiciones experimentales.

Para los experimentos con datos sintéticos se generan dos tipos de conjuntos: uno linealmente separable y otro no linealmente separable. En ambos casos se entrena una regresión logística y se evalúa el error medio absoluto (MAE), definido como

$$e_{\text{MAE}} = \frac{1}{N} \sum_{i=1}^N |t_i - \tilde{t}_i|, \quad (1)$$

donde t_i es la etiqueta esperada y $\tilde{t}_i$ es la salida estimada por el modelo.

Para los experimentos con textos, se utiliza FEINA como conjunto base. Cada par contiene un segmento complejo y una propuesta simplificada; por tanto, el problema se formula como clasificación binaria. En los modelos basados en regresión logística, el texto se transforma primero a una representación numérica. En los modelos basados en LLMs, el texto se clasifica directamente mediante instrucciones en lenguaje natural.

3. Perceptrón

El perceptrón fue propuesto por Rosenblatt en 1958 como un modelo probabilístico inspirado en procesos de almacenamiento y organización de información en el cerebro [2]. En aprendizaje supervisado, puede interpretarse como un clasificador lineal binario: aprende un vector de pesos $\vec{w}$ que define una frontera de decisión en el espacio de entrada. Dado un vector $\vec{x}$, el modelo calcula el producto interno $\vec{x}^T \vec{w}$ y asigna una clase según el signo del resultado:

$$\hat{y} = \text{signo}(\vec{x}^T \vec{w}). \quad (2)$$

La función signo convierte la salida continua del producto interno en una decisión discreta. En este contexto, si $\vec{x}^T \vec{w} > 0$, la muestra se asigna a la clase positiva; si $\vec{x}^T \vec{w} < 0$, se asigna a la clase negativa. El caso exactamente igual a cero representa una muestra ubicada sobre la frontera de decisión y suele resolverse por convención de implementación [3].

Esta formulación es simple y eficiente, pero impone una restricción fuerte: el modelo solo puede separar perfectamente problemas linealmente separables. Si las clases se encuentran mezcladas de forma no lineal, como ocurre en estructuras similares al problema XOR (fig.1), una única frontera lineal no puede resolver correctamente todas las muestras. Esta limitación es relevante para entender por qué el experimento sintético incluye un conjunto no linealmente separable [3].

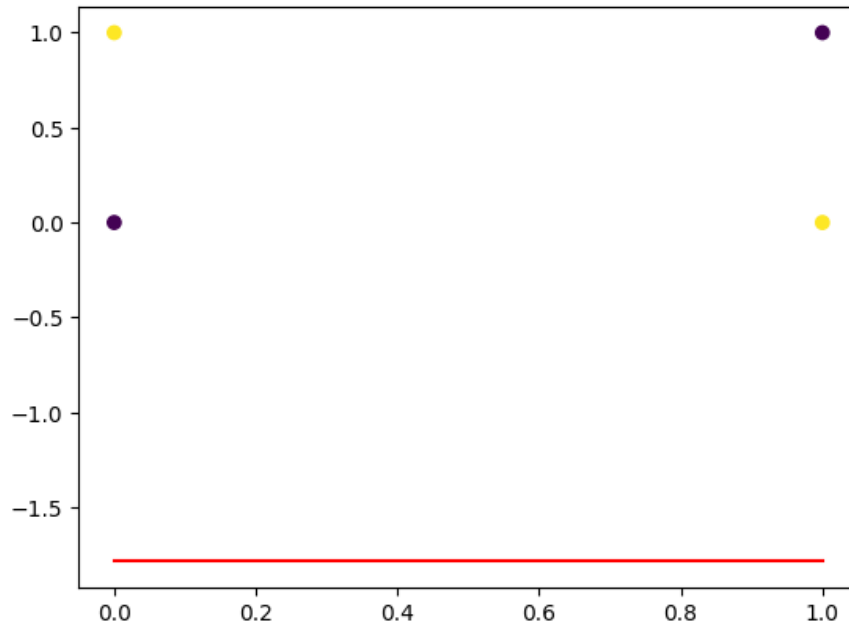


Figura 1: Ejemplo conceptual de cómo el XOR no es linealmente separable. El perceptrón para generar esta gráfica fue entrenado por 6000 iteraciones con un alfa de 1.

En esta figura se observa la imposibilidad de trazar una línea recta que separe completamente las dos clases: los puntos amarillos (salida 1 en XOR) de los puntos morados (salida 0 en XOR), dejando todos los puntos de una clase a un lado y los de la otra al lado opuesto.

El entrenamiento del perceptrón actualiza los pesos usando únicamente las muestras mal clasificadas. Si U es el conjunto de dichas muestras, una actualización típica puede escribirse como.

$$E = - \sum_{X_i \in U} \vec{w}^T X_i t_i \quad (3)$$

$$\nabla E = \sum_{X_i \in U} t_i X_i \quad (4)$$

$$\vec{w}_{k+1} = \vec{w}_k - \alpha \nabla E \quad (5)$$

donde α es la tasa de aprendizaje y E la función de error, donde los $X_i \in U$ son las muestras mal clasificadas y t_i con las etiquetas reales. El teorema de convergencia del perceptrón establece que, si los datos son linealmente separables, existe un margen positivo y se usa una tasa de aprendizaje constante, el algoritmo encuentra una solución separadora en un número finito de actualizaciones. La intuición de la demostración es que cada actualización aumenta la alineación entre los pesos y algún vector separador óptimo, mientras que el crecimiento de la norma de los pesos queda acotado; ambas condiciones implican que no puede haber infinitos errores si existe una separación lineal.

En el perceptrón clásico, usar $\alpha = 1$ es suficiente porque la tasa de aprendizaje solo escala el tamaño de las actualizaciones, pero no ajusta la dirección. Para datos linealmente separables, cambiar α modifica la escala de los pesos, pero no necesariamente la orientación de la frontera final. En problemas no separables o en variantes con criterios de parada prácticos, la elección de α sí puede afectar la estabilidad del entrenamiento y el comportamiento observado [4].

4. Regresión logística

La regresión logística es un modelo estadístico de clasificación binaria desarrollado formalmente para analizar variables de respuesta dicotómicas. Una formulación influyente fue presentada por Cox en 1958 para secuencias binarias [5]. En lugar de producir directamente una clase discreta, el modelo estima la probabilidad de pertenencia a una clase mediante una función sigmoide aplicada sobre una combinación lineal de las entradas:

$$\hat{y} = \sigma(\vec{x}^T \vec{w}) = \frac{1}{1 + e^{-\vec{x}^T \vec{w}}}. \quad (6)$$

La salida $\hat{y}$ pertenece al intervalo $[0, 1]$, por lo que puede interpretarse como una probabilidad. Para convertirla en una clase se usa un umbral, usualmente 0,5. Esta diferencia respecto al perceptrón es importante porque la sigmoide es diferenciable y permite ajustar los pesos mediante gradiente. Además, la regresión logística es ampliamente usada en problemas de clasificación binaria por su interpretabilidad y eficiencia [6].

Matemáticamente, la sigmoide suaviza la decisión. Valores muy negativos de $\mathbf{x}^T \mathbf{w}$ se aproximan a 0, valores muy positivos se aproximan a 1, y valores cercanos a cero quedan cerca de 0,5. Por eso, el modelo no solo indica una clase, sino también qué tan cerca está una muestra de la frontera de decisión.

4.1. Diferencias respecto al perceptrón

La implementación de regresión logística se construye usando como base la estructura del perceptrón, pero modifica tres componentes esenciales: la función de activación, el cálculo del gradiente y la función de error. Estas diferencias no son solo detalles de implementación; cambian la naturaleza del entrenamiento.

Diferencias principales entre perceptrón y regresión logística		
Componente	Perceptrón	Regresión logística
Salida	Clase discreta obtenida con la función signo.	Probabilidad en $[0, 1]$ obtenida con sigmoide.
Activación	No diferenciable en la frontera.	Diferenciable, útil para optimización por gradiente.
Uso de muestras	Actualiza con muestras mal clasificadas.	Todas las muestras aportan al gradiente.
Error	Criterio basado en errores de clasificación.	Error continuo entre etiqueta y probabilidad estimada.
Interpretación	Separador lineal binario.	Separador lineal probabilístico.

Tabla 1: Tabla con principales diferencias entre el algoritmo de perceptrón y regresión logística

En el perceptrón, la función de activación puede representarse como:

Algorithm 1 Activación del perceptrón

```
1: function ACTIVACIONPERCEPTRON( $X, w$ )  
2:   return signo( $Xw$ )  
3: end function
```

En la regresión logística, esta activación se reemplaza por:

Algorithm 2 Activación de regresión logística

```
1: function ACTIVACIONLOGISTICA( $X, w$ )  
2:   return sigmoide( $Xw$ )  
3: end function
```

El gradiente implementado para la regresión logística sigue la forma vectorizada:

$$E = -\frac{1}{N} \sum_{i=1}^N \hat{y}_i^{t_i} (1 - \hat{y}_i)^{1-t_i} \quad (7)$$

$$\nabla E = -\frac{1}{N} \sum_{i=1}^N x_i (t_i - \hat{y}_i). \quad (8)$$

Esta forma aparece al medir la discrepancia entre la etiqueta real t_i y la probabilidad estimada $\hat{y}_i$ respecto al vector de pesos actual. Si el modelo predice una probabilidad menor que la etiqueta real, el término $(t_i - \hat{y}_i)$ empuja los pesos en dirección de x_i ; si predice una probabilidad demasiado alta, el término tiene signo contrario. Así, cada muestra aporta información proporcional al error cometido, no solo una señal binaria de correcto o incorrecto.

La actualización de pesos resultante es:

$$\tilde{\mathbf{w}}_{k+1} = \vec{w}_k - \alpha \nabla. \quad (9)$$

Esta formulación es más informativa que la actualización del perceptrón para problemas con incertidumbre o solapamiento entre clases. El perceptrón reacciona únicamente cuando una muestra queda mal clasificada; la regresión logística, en cambio, ajusta los pesos incluso cuando la clasificación discreta es correcta pero la probabilidad estimada todavía está lejos de la etiqueta. Por eso puede producir curvas de error más suaves y análisis más detallados durante el entrenamiento.

4.2. Condición de Parada Temprana (Early Stopping)

Para la implementación de la regresión logística, se utilizó el método de la Parada Temprana (Early Stopping). La historia del concepto se remonta a 1970 en donde se trabaja con el algoritmo de Landweber, para resolver problemas inversos mal planteados [7]. “Early Stopping” es una técnica que busca evitar el sobreajuste (Overfitting), por lo general, se implementa sobre un conjunto de validación, el cual es independiente tanto del conjunto de entrenamiento como del de prueba [8]. La principal ventaja de implementar “Early Stopping” es la disminución obtenida en el costo computacional [7].

Para la implementación de la regresión logística se optó por un método numérico llamado criterio de convergencia por cambio en la solución. Debido a las propiedades del cálculo diferencial, se sabe que la derivada de una función tiende a cero cuando el algoritmo se aproxima a un mínimo local. Por esta razón, se define un valor umbral ϵ y se detiene el entrenamiento cuando la magnitud de la derivada respecto a w es menor que dicho valor.

$$||\nabla_w f(w)|| < \epsilon \quad (10)$$

Cabe recalcar que este criterio de convergencia solo se evalúa después de las primeras 100 iteraciones, con el objetivo de evitar una finalización prematura del entrenamiento. El método implementado es de la forma:

Algorithm 3 Criterio de convergencia

```

1: function EARLYSTOP( $\delta$ ,  $it$ ,  $\epsilon$ )
2:   if  $it > 100$  and  $||\delta|| < \epsilon$  then
3:     return True
4:   else
5:     return False
6:   end if
7: end function

```

donde δ es la derivada, it el número de iteración y ϵ el umbral seleccionado. Es importante mencionar que, por razones de rendimiento computacional, además de implementar el criterio de Early Stopping, se estableció un número máximo de iteraciones para cada entrenamiento. De esta manera, se garantiza que el proceso de optimización finalice en un tiempo razonable incluso en los casos donde el criterio de convergencia no se satisfaga.

5. Resultados con datos sintéticos

Esta sección evalúa la regresión logística en dos escenarios controlados: datos linealmente separables y datos no linealmente separables. El objetivo no es solamente comparar errores, sino observar cómo la forma geométrica del conjunto de datos afecta la frontera de decisión aprendida.

5.1. Datos linealmente separables

Para generar los datos linealmente separables se utiliza la función `make_blobs` de `scikit-learn`, configurada con dos centros, una desviación de agrupamiento de 0,1 para generar 100 puntos de dos dimensiones. Esta elección produce dos nubes de puntos compactas, por lo que una frontera lineal debería separar las clases con bajo error.

En este caso, se espera que el error disminuya de forma clara durante el entrenamiento y que la superficie de decisión quede ubicada entre los dos grupos de puntos. Para el entrenamiento se utilizó el 70 % de las muestras generadas y el resto se designó para graficarlos junto con la superficie de decisión generada con el entrenamiento.

Para graficar dicha superficie, se parte de la ecuación lineal aprendida por el modelo:

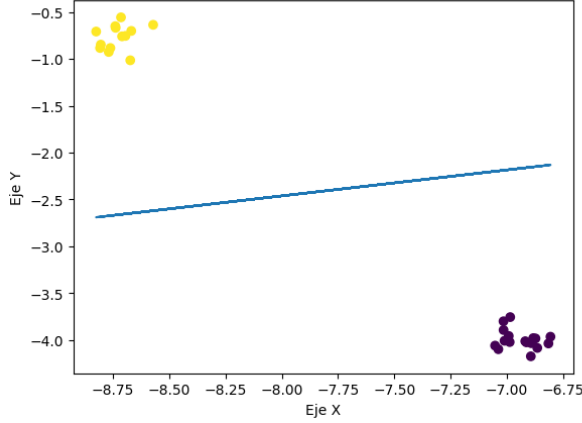
$$w_0 + w_1x_1 + w_2x_2 = 0. \quad (11)$$

Despejando x_2 , se obtiene:

$$x_2 = -\frac{w_0 + w_1x_1}{w_2}. \quad (12)$$

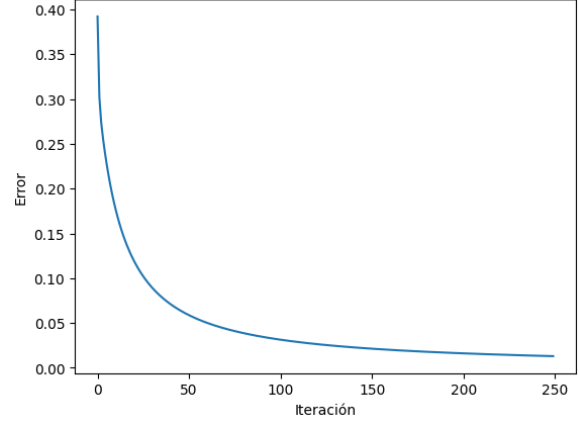
Esta recta se superpone al diagrama de dispersión para visualizar la separación inducida por los pesos aprendidos.

Diagrama de Dispersión Linealmente separable con la superficie de decisión



(a) Superficie de decisión.

Linealmente separable - Gráfica de Error



(b) Error por iteración.

Figura 2: Resultados visuales para datos linealmente separables. El entrenamiento se realizó con un máximo de 250 iteraciones. En la imagen izquierda, se tiene el diagrama de dispersión donde los puntos amarillos pertenecen a la clase 1 y los puntos morados a la clase 0, esto corresponde al 30 % de los datos generados. La recta azul es la obtenida tras el cálculo mostrado en la ecuación 12. Se puede observar que la superficie de decisión separa de forma correcta estas muestras que no se usaron para el entrenamiento, y que se encuentra a una distancia media de ambas clases. La imagen de la derecha, muestra la gráfica de error del modelo a lo largo de las 250 iteraciones de entrenamiento, la cual decrece rápidamente en las primeras iteraciones para estabilizarse aproximadamente en la iteración 50.

5.2. Datos no linealmente separables

Para generar los datos no linealmente separables se utiliza la función `make_moons` de `scikit-learn`, con un ruido agregado de 0,15, para generar de igual forma 100 muestras de dos dimensiones. Esta función produce dos clases con forma de medias lunas entrelazadas, una estructura que no puede separarse completamente con una única recta.

En este caso, una frontera lineal resulta insuficiente para separar por completo las clases. Por tanto, aunque el entrenamiento puede reducir el error, se espera que el MAE final sea mayor que en el caso linealmente separable. Este resultado no indica necesariamente una falla de implementación; más bien evidencia una limitación esperada del modelo lineal.

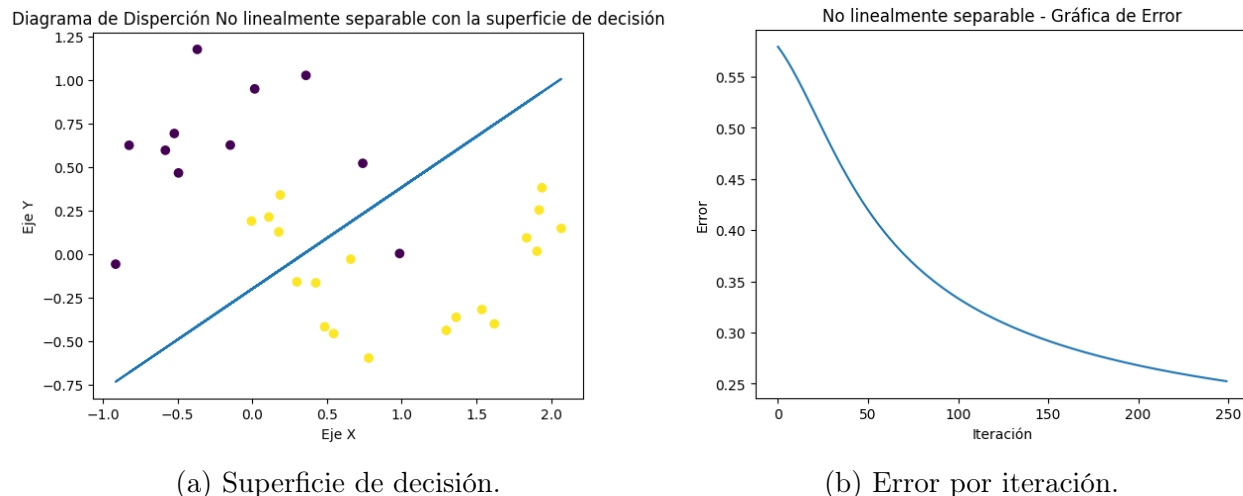


Figura 3: Resultados visuales para datos no linealmente separables. El entrenamiento se realizó con un máximo de 250 iteraciones. En la imagen izquierda, se tiene el diagrama de dispersión donde los puntos amarillos pertenecen a la clase 1, los puntos morados a la clase 0 y representan el 30 % de las muestras generadas, el resto se utilizó para entrenar. La recta azul es la obtenida tras el entrenamiento y el cálculo mostrado en la ecuación 12. En este caso un modelo lineal no es suficiente para separar los datos, por lo que una parte queda al lado contrario de la superficie de decisión. La imagen de la derecha, muestra la gráfica de error del modelo a lo largo de las 250 iteraciones de entrenamiento, la cual se asemeja más a una línea, lo que quiere decir que al modelo se le dificulta más clasificar los datos.

5.3. Resultados del entrenamiento

A continuación se presenta los datos obtenidos de entrenar el modelo de regresión logística con los datos no y linealmente separables. El entrenamiento se llevó a cabo con el criterio explicado en la sección 4.2, con un máximo de 250 iteraciones y con el 70 % del dataset generado.

Tabla comparativa del entrenamiento con datos no y linealmente separables

Iteración	Linealmente separable	No linealmente separable	Diferencia Absoluta	Menor error
0	0.556348	0.460561	0.095787	No separable
24	0.084619	0.405352	0.320733	Separable
49	0.052553	0.361052	0.308499	Separable
74	0.037930	0.330211	0.292281	Separable
99	0.029623	0.308413	0.278790	Separable
124	0.024282	0.292426	0.268143	Separable
149	0.020565	0.280268	0.259703	Separable
174	0.017831	0.270728	0.252897	Separable
199	0.015737	0.263047	0.247311	Separable
224	0.014081	0.256728	0.242647	Separable
249	0.012741	0.251438	0.238697	Separable

Tabla 2: La tabla muestra la evolución del MAE durante el entrenamiento a lo largo de 250 iteraciones, registrando los resultados cada 25 iteraciones, tanto para datos linealmente separables como para los que no lo son. Además la tercera columna presenta la diferencia absoluta entre los MAE del entrenamiento de ambos datasets. La última columna presenta el dataset con menor MAE. Se puede observar que los datos linealmente separables se muestran durante la mayoría de iteraciones con un error menor. Solo en la iteración 0 los datos no linealmente separable se muestran con un MAE menor, lo cual se debe a la aleatoriedad en la selección del vector inicial de pesos. También la segunda columna presenta un decrecimiento mucho más pronunciado que la tercera.

5.4. Resultados promedio para 10 corridas

En esta sección se reporta el promedio del MAE y su desviación estándar en 10 corridas, generando datos aleatorios en cada una y designando el 70 % para entrenamiento y el resto para evaluación. La implementación registra el promedio del MAE obtenido sobre los datos de prueba, la desviación estándar de dicho MAE y el valor mínimo de MAE alcanzado durante la evaluación sobre el conjunto de prueba.

Comparación de los resultados obtenidos en las 10 iteraciones de los datos no y linealmente separables

	Linealmente Separable	No linealmente Separable
Promedio de MAE	0.020643	0.263950
Promedio de Desviación estándar	0.036780	0.028561
Mínimo error MAE obtenido	0.000588	0.226277

Tabla 3: Resumen de los resultados obtenidos con regresión logística utilizando datos sintéticos a lo largo de 10 corridas independientes. La primera fila muestra el MAE promedio sobre el conjunto de prueba, calculado a partir de las 10 corridas. La segunda fila corresponde a la desviación estándar del MAE en el conjunto de prueba, mientras que la última fila presenta el valor mínimo de MAE obtenido durante las evaluaciones sobre dicho conjunto. El promedio MAE y mínimo MAE presentan una gran diferencia entre las corridas con datos separables y no separables. Principalmente, los datos separables presentan un error de evaluación relativamente bueno en comparación con los no separables. Por otro lado, la desviación estándar es muy similar en ambos, esto puede deberse a que el descenso gradiente se comporta de forma similar con ambos tipos de datos. Se puede derivar el valor de la precisión dado el MAE, debido a que se está trabajando sobre clasificación binaria, entonces es su complemento

6. Regresión logística con TF-IDF

TF-IDF (*Term Frequency-Inverse Document Frequency*) es una representación clásica para documentos. Su objetivo es asignar mayor peso a términos frecuentes en un documento, pero menos comunes en el corpus completo. De esta manera, palabras muy generales pierden relevancia, mientras que términos más distintivos adquieren mayor peso [9].

En este proyecto, TF-IDF se usa como etapa de extracción de características. La implementación utiliza `TfidfVectorizer` de `scikit-learn` con un analizador personalizado. Este analizador aplica el preprocesamiento definido en el proyecto antes de construir el vocabulario y calcular los pesos TF-IDF. Luego, la matriz dispersa producida por el vectorizador se transforma con `toarray()` y se convierte a un tensor `torch.float32`, que es el formato esperado por la regresión logística implementada previamente.

6.1. Preprocesamiento

Antes de fijar una configuración para TF-IDF se realizó una comparación exploratoria entre varias estrategias de preprocesamiento. El objetivo no era solamente obtener una métrica, sino entender qué decisiones lingüísticas afectaban la representación del texto financiero. En particular, se compararon tokenizadores basados en NLTK, expresiones regulares, spaCy y Stanza; además, se probaron lematizadores basados en WordNet, Simplemma, spaCy y Stanza.

Una decisión importante fue probar expresiones regulares con y sin números. Para ello se usaron dos patrones principales: `[^\W_]+`, que conserva secuencias alfanuméricas, y `[a-zAÉÍÓÚñü]+`, que conserva solamente letras. Esta diferencia es relevante porque los textos financieros contienen porcentajes, montos, fechas y referencias numéricas; sin embargo, también existe el

riesgo de que los números introduzcan ruido o vocabulario demasiado específico para un conjunto de datos pequeño. Por esta razón, no se asumió de antemano que conservar números fuera mejor: se evaluaron ambas opciones.

El flujo de preprocesamiento fue: tokenizador $\rightarrow$ stopwords $\rightarrow$ lematizador. Es por ello que se probaron las combinaciones de tokenizar (regex, nltk, etc), con las diferentes formas de eliminar stopwords (nltk, spacy, etc) y cada lematizador, para poder comparar qué configuración tenía el mejor resultado.

Para evitar que la comparación dependiera de una única partición del conjunto de datos, cada alternativa se evaluó con varias semillas (101, 202, 303). En cada semilla se entrenó la misma regresión logística y una tasa de aprendizaje fija. De esta forma, la comparación distingue mejor entre la calidad de la representación textual y no una mala elección accidental de la tasa de aprendizaje. Adicional, se prueba por una cantidad de 500 iteraciones con un subconjunto de FEINA de 30 datos, separados en 15 textos complejos y 15 simples.

Estadísticas de tokenización por configuración de preprocesamiento			
Configuración	Vocabulario	Prom. tokens	Máx. tokens
wordtokenizer + nltk + wordnet	325	28.467	45
regex numérico + nltk + wordnet	321	23.733	38
regex no numérico + nltk + wordnet	321	23.733	38
regex numérico + nltk + simplemma	273	23.733	38
regex no numérico + nltk + simplemma	273	23.733	38
spacy numérico + spacy + spacy	237	24.600	40
spacy no numérico + spacy + spacy	233	19.867	32
stanza numérico + nltk + stanza	276	24.467	37
stanza no numérico + nltk + stanza	276	24.467	37

Tabla 4: En la tabla se observan los resultados obtenidos de la comparación de los resultados entre cada configuración. Donde el vocabulario hace referencia a la cardinalidad del conjunto de tokens y las columnas de promedio de tokens y máximo de tokens, podemos ver datos de importancia para saber cuál configuración es la que usa menos cantidad de tokens.

Rendimiento por configuración de preprocesamiento					
Configuración	MAE α fijo	Precisión α fijo	α calibrado	MAE α calibrado	Precisión α calibrado
wordtokenizer + nltk + wordnet	0.4194	0.500000	0.016444	0.417308	0.833333
regex numérico + nltk + wordnet	0.412923	0.666667	0.017145	0.413215	0.833333
regex no numérico + nltk + wordnet	0.412923	0.666667	0.017145	0.413215	0.833333
regex numérico + nltk + simplemma	0.429078	0.500000	0.016946	0.423593	0.833333
regex no numérico + nltk + simplemma	0.429078	0.500000	0.016946	0.423593	0.666667
spacy numérico + spacy + spacy	0.417931	0.666667	0.016983	0.424317	0.833333
spacy no numérico + spacy + spacy	0.407284	0.833333	0.016144	0.412773	0.833333
stanza numérico + nltk + stanza	0.398137	0.666667	0.016896	0.362858	1.000000
stanza no numérico + nltk + stanza	0.370942	0.833333	0.016896	0.394815	0.833333

Tabla 5: En la tabla se muestra el MAE y la precisión de acuerdo a un α fijo de $\alpha = 0,01$ y a uno calibrado, luego de 500 iteraciones. Se usan resultados basados en una calibración de α utilizando optuna y una cantidad total de 5 intentos, sin embargo para el desarrollo final del proyecto decidimos utilizar un α fijo para la comparación justa con otras implementaciones.

Para la calibración se utilizó un rango acotado de valores de α definido a partir de la constante de Lipschitz del gradiente de la función objetivo. Esta elección se basa en resultados clásicos de optimización convexa: cuando el gradiente de una función es Lipschitz continuo con constante L , un tamaño de paso $\alpha \leq 1/L$ proporciona una cota segura para el descenso de gradiente. En este contexto, dicha cota no se interpreta como el valor óptimo de α , sino como un límite teóricamente razonable para restringir la búsqueda y evitar valores demasiado grandes que puedan producir inestabilidad o divergencia [10].

Por esta razón, la calibración de α se realizó dentro de un intervalo acotado por la constante de Lipschitz. De esta forma, la selección final del parámetro combina una restricción teórica, derivada del comportamiento del gradiente, con una elección empírica basada en el desempeño observado del modelo.

Para el reporte final del entrenamiento de regresión logística con TF-IDF escogimos la configuración de Stanza, específicamente tokenizador Stanza sin números, stopwords de NLTK y lematizador de Stanza, ya que esta configuración obtuvo una mejor precisión en promedio. Esto no quiere decir que otra configuración no sea válida, ya que vemos resultados similares entre varias configuraciones y algunas de ellas como regex + simplemma pueden mejorar el rendimiento al ser un procesamiento menos costoso, más rápido y aún así nos da resultados aceptables, es decir, cercanos a los resultados de sus competidores pero a una cuota de rendimiento más baja que en muchas ocasiones es preferible.

6.1.1. Análisis de la lematización y la extracción de “stopwords”

Para los siguientes ejemplos

Texto: El equipo realizó la redacción del libro luego de obtener la información anterior y reunir las fuentes bibliográficas y virtuales respectivas.	Resultado: [‘equipo’, ‘realizar’, ‘redacción’, ‘libro’, ‘luego’, ‘obtener’, ‘información’, ‘anterior’, ‘reunir’, ‘fuente’, ‘bibliográfico’, ‘virtual’, ‘respectivo’]
--	--

Figura 4: Se puede observar que el preprocesador elimina preposiciones como ‘de’, cambia las conjunciones a verbos como ‘realizó’ a ‘redactar’. También cambia de plural a singular y elimina signos de puntuación como ‘.’.

Texto: Luego de haber obtenido la anterior información y haber hecho acopio de las fuentes bibliográficas y virtuales respectivas, el equipo procedió a realizar la redacción o construcción del libro.	Resultado: [‘luego’, ‘haber’, ‘obtener’, ‘anterior’, ‘información’, ‘haber’, ‘hacer’, ‘acopio’, ‘fuente’, ‘bibliográfico’, ‘virtual’, ‘respectivo’, ‘equipo’, ‘proceder’, ‘realizar’, ‘redacción’, ‘construcción’, ‘libro’]
--	--

Figura 5: Los conectores como ‘y’ y ‘o’ se eliminan, se destaca que la conjugación ‘hecho’ se convierte de manera correcta a ‘hacer’.

6.2. Construcción de descriptores

Una vez preprocesados los documentos, `TfidfVectorizer` aprende el vocabulario del corpus y construye una matriz documento-término. Cada fila representa un documento y cada columna representa un término del vocabulario aprendido. El valor de cada celda indica la importancia relativa del término en ese documento.

A continuación, se presenta un análisis cualitativo de esta representación, se selecciona textos simples y complejos y se reportan los términos con mayor peso TF-IDF. Esto permite verificar si el descriptor captura vocabulario relevante para la complejidad o si queda dominado por términos poco informativos.

6.2.1. Análisis de la transformación TF-IDF

Para el análisis de la transformación TF-IDF se tomaron los documentos:

- **Documento 0:** Luego de haber obtenido la anterior información y haber hecho acopio de las fuentes bibliográficas y virtuales respectivas, el equipo procedió a realizar la redacción o construcción del libro.
- **Documento 1:** El equipo realizó la redacción del libro luego de obtener la información anterior y reunir las fuentes bibliográficas y virtuales respectivas.

Valores TF-IDF de la transformación		
	Documento 0	Documento 1
acopio	0.266549	0
anterior	0.189652	0.267497
bibliográfico	0.189652	0.267497
construcción	0.266549	0
equipo	0.189652	0.267497
fuentes	0.189652	0.267497
haber	0.533098	0
hacer	0.266549	0
información	0.189652	0.267497
libro	0.189652	0.267497
luego	0.189652	0.267497
obtener	0.189652	0.267497
proceder	0.266549	0
realizar	0.189652	0.267497
redacción	0.189652	0.267497
respectivo	0.189652	0.267497
reunir	0	0.375958
virtual	0.189652	0.267497

Tabla 6: En esta tabla se puede observar que hay ciertas palabras que se encuentran en ambos documentos. Además palabras como ‘acopio’, ‘construcción’, ‘haber’, ‘hacer’, ‘proceder’ y ‘reunir’ se les asigna un peso mayor al solo presentarse en alguno de los dos documentos. De hecho, el verbo ‘haber’ solo se presenta en el documento 0 pero lo hace en dos ocasiones, obteniendo un valor mucho mayor. A su vez, se puede observar que el resto de palabras que se encuentran en ambos documentos, presentan un menor peso en proporción a la cantidad de tokens en cada documento.

6.3. Clasificación y resultados

El conjunto FEINA se procesa usando sus columnas **Segment** y **Proposal**. En la implementación actual, cada **Segment** se agrega al corpus con etiqueta 1, y cada **Proposal** se agrega con etiqueta 0. Así, cada par del archivo aporta dos muestras al clasificador: una compleja (clase 1) y una simple (clase 0).

Después de construir la matriz TF-IDF, se entrena con la regresión logística usando el 70 % de los documentos y con el resultado del entrenamiento se evalúa el resto (30 %) de

documentos.

Métrica	Valor
MAE promedio	0.48039379715919495
Precisión promedio	0.5740275979042053

Tabla 7: Resultados de regresión logística con TF-IDF con la evaluación del 30 % del dataset.

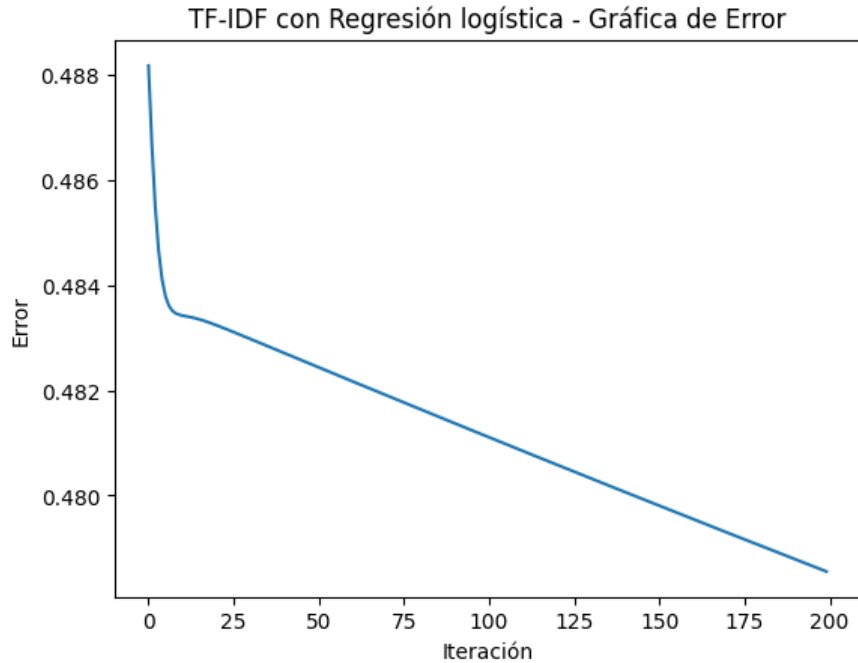


Figura 6: Gráfica de aprendizaje del entrenamiento con el 70 % del dataset por 200 iteraciones y $\alpha = 1$. En el eje X se muestra el número de iteración y en el eje Y el valor MAE. La gráfica primero tiene un decrecimiento pronunciado y después se asemeja a una línea, y el rango del error $[0,48, 0,488]$ indica que el modelo no tiene mucha certeza.

Los resultados presentados en la tabla 7 muestran que, con el dataset completo de 10624 entradas, donde 5312 son etiquetas complejas y 5312 son etiquetas simples, tiene una precisión promedio de $\approx 0,56$ que nos indica que tiene un rendimiento limitado, ya que apenas supera el 50 % de aciertos, haciendo que no podamos notar un rendimiento mucho más alto que únicamente clasificar al azar.

7. Regresión logística con vectores empotrados

Los vectores empotrados representan textos en un espacio numérico denso (espacio vectorial), donde la cercanía geométrica intenta reflejar similitud semántica. A diferencia de TF-IDF, que produce vectores dispersos basados en conteos ponderados, los embeddings capturan información distribuida aprendida previamente a partir de grandes corpus. [11]

En este trabajo se utilizan esos vectores con la misma regresión logística implementada en la primera parte. Esta separación mantiene constante el clasificador y permite analizar si las diferencias provienen de la representación del texto.

La comparación incluyó cuatro modelos:

- `sentence-transformers/distiluse-base-multilingual-cased-v1`
- `bert-base-multilingual-cased`
- `dccuchile/bert-base-spanish-wwm-cased` (BETO)
- `Buzzeitor/roberta-base-bne` (RoBERTa)

La intención fue probar distintas formas razonables de obtener vectores densos para los textos y observar cuál funcionaba mejor al entrenar la misma regresión logística.

7.1. Proyección de textos

La proyección transforma cada documento en un vector de dimensión fija. La idea central es que textos con significado parecido queden representados por vectores cercanos, incluso si no comparten exactamente las mismas palabras [11]. Esto es útil para el proyecto porque la complejidad textual no depende solo de términos aislados: también influyen la forma de expresar una idea, la densidad conceptual y el contexto semántico de la oración.

Esta representación se evaluó como una alternativa a TF-IDF. Mientras TF-IDF se basa en frecuencia de términos y puede funcionar muy bien cuando ciertas palabras distinguen clases, los embeddings permiten aprovechar conocimiento aprendido previamente por modelos entrenados en grandes corpus. Por ejemplo, dos textos financieros pueden usar vocabulario distinto pero expresar una idea parecida; una representación densa puede capturar parte de esa relación mejor que una bolsa de palabras. Por eso ambos enfoques se comparan: no porque uno sea una extensión directa del otro, sino porque representan dos maneras distintas de alimentar el mismo clasificador.

Una forma de realizar esto es con bibliotecas como BERT (Bidirectional Encoder Representations from Transformers, por sus siglas en inglés), que es una variante bidireccional de las “Transformers networks”, entrenada para predecir conjuntamente si una palabra enmascarada de su contexto y para clasificar si dos oraciones son consecutivas o no [12].

A partir de BERT han surgido variantes entrenadas o adaptadas para distintos idiomas y corpus. En el caso del español, BETO puede entenderse como una versión de BERT entrenada específicamente con textos en español, por lo que sus representaciones están mejor ajustadas a las características del idioma. De forma similar, RoBERTa es una variante de la familia BERT que modifica algunos aspectos del entrenamiento original para mejorar la calidad de las representaciones; en este trabajo se utiliza una versión entrenada para el español. Por esta

razón, además del modelo multilingüe, se comparan BETO y RoBERTa como alternativas especializadas para representar textos en español.

Por esta razón se probaron varias alternativas de embeddings. El modelo “distiluse-base-multilingual-cased-v1” fue relevante porque está diseñado para producir embeddings de oración directamente. BERT, BETO y RoBERTa se usaron como alternativas contextuales basadas en transformadores, con el fin de revisar si otra representación densa podía producir mejores resultados con la misma regresión logística.

Modelos evaluados para generar vectores empotrados.	
Modelo	Dimensiones
distiluse-base-multilingual-cased-v1	512
bert-base-multilingual-cased	768
dccuchile/bert-base-spanish-wwm-cased	768
Buzzeitor/roberta-base-bne	768

Tabla 8: Dimensiones de cada modelo utilizado para la comparación

Para la siguiente comparación se utilizó el mismo principio de comparación de TF-IDF, se estableció un α fijo de $\alpha = 0,01$ y se evaluó el modelo por 500 iteraciones con 30 muestras por iteración, para comparar justamente los modelos, además se realizaron corridas con 3 semillas distintas (101, 202, 303) y se registraron los datos de la mejor corrida (menor MAE, mayor precisión). Para la calibración se usaron 5 corridas y un rango acotado de α definido por la técnica de Lipschitz.

Modelo	MAE fijo	Accuracy fijo	Alpha calibrado	MAE calibrado	Accuracy calibrado
DistilUSE Multilingual	0.569139	0.333333	0.003704	0.563137	0.333333
BERT Multilingual	0.504664	0.333333	0.248051	0.469454	0.666667
BETO Español	0.486096	0.666667	0.253731	0.405959	0.833333
RoBERTa Español	0.502260	0.500000	0.002460	0.502018	0.500000

Tabla 9: Resultados de regresión logística con vectores empotrados

Aunque DistilUSE Multilingual no obtuvo los mejores resultados cuantitativos en términos de MAE y accuracy, se seleccionó como modelo base para la regresión logística con embeddings debido a su simplicidad de integración, menor costo computacional y capacidad multilingüe. Este modelo permite generar representaciones vectoriales directamente comparables entre textos sin requerir un proceso de ajuste fino, lo que facilita su incorporación en un pipeline ligero y reproducible.

En este sentido, DistilUSE Multilingual se considera una alternativa adecuada cuando se prioriza eficiencia, rapidez de inferencia y simplicidad del sistema por encima del máximo desempeño predictivo. No obstante, los resultados muestran que modelos especializados en español, particularmente BETO Español, alcanzan un rendimiento superior, por lo que constituyen una opción más recomendable si el objetivo principal es maximizar la precisión del sistema.

7.2. Clasificación y resultados

Resultados de regresión logística con vectores empotrados	
Métrica	Valor
MAE	0.477526068687438
Precisión	0.576537013053894
Dimensión del vector	512

Tabla 10: En la tabla se muestra los resultados de la evaluación de la regresión logística, con el 30 % del dataset, alimentada con los vectores empotrados del **SentenceTransformer**. En la primera columna se encuentran las métricas evaluadas (MAE, precisión y dimensión del vector) y en la segunda columna se presentan los valores obtenidos. La superioridad del modelo BERT frente al TF-IDF queda reflejada en su mayor precisión y menor valor MAE, aunque no hay una diferencia significativa.

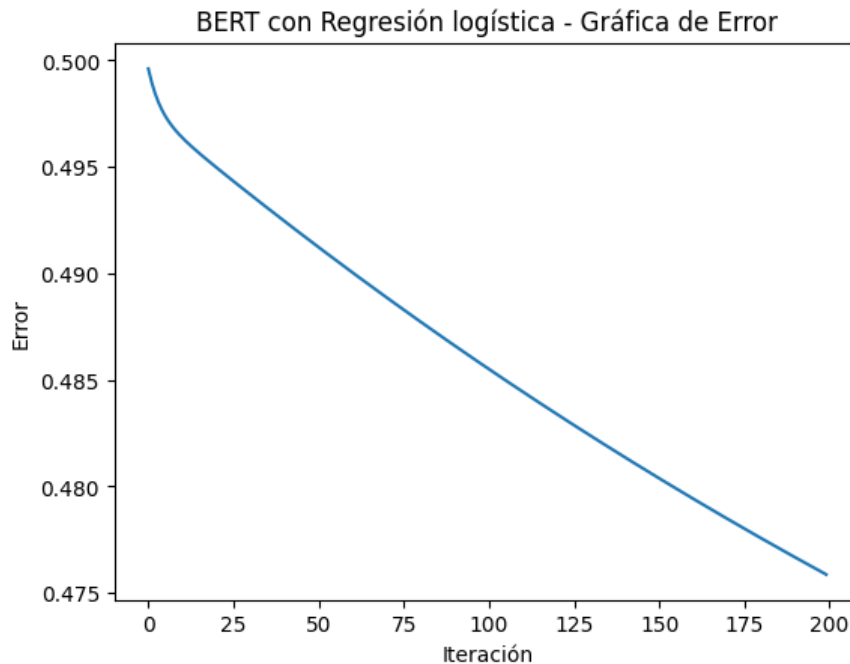


Figura 7: Gráfica de aprendizaje del entrenamiento con el 70 % del dataset por 200 iteraciones. En el eje X se muestra el número de iteración y en el eje Y el valor MAE. La gráfica se asemeja a una línea, y el rango del error $[0,475, 0,5]$ indica que el modelo no tiene mucha certeza, aunque es un poco mejor que el modelo con TF-IDF.

8. Modelos de lenguaje grandes (LLMs)

Los modelos de lenguaje grandes o *Large Language Models* (LLMs) permiten abordar la clasificación de textos mediante instrucciones en lenguaje natural. En lugar de entrenar explícitamente un clasificador sobre características numéricas, se formula un prompt que describe

la tarea y solicita una etiqueta. Esta aproximación reduce la ingeniería de características, pero introduce otros retos: sensibilidad al prompt, costo computacional, consumo de memoria, tiempo de inferencia y dificultad para controlar la salida exacta.

En este trabajo se evalúan dos LLMs de Hugging Face. Ambos reciben instrucciones para clasificar cada texto como simple o complejo usando las clases 0 y 1, respectivamente. Para que la comparación sea justa, los modelos deben usar el mismo conjunto de textos, el mismo formato de salida esperado y, en el caso few-shot, los mismos ejemplos demostrativos.

En este apartado se utilizó como hardware una GPU - Nvidia RTX 3060 de 8Gb de VRAM, para el procesamiento más pesado, ya que los servidores de Colab están restringidos a una cantidad limitada de horas por ejecución.

8.1. SmolLM3

SmolLM3 es un modelo instruccional de Hugging Face usado como una alternativa relativamente compacta para clasificación mediante prompts. En este proyecto se utiliza con un prompt que define el rol del modelo como clasificador de textos financieros, especifica las clases válidas y exige una respuesta en formato `ID:CLASE`. La elección de este formato permite evaluar automáticamente lotes de textos sin interpretar respuestas largas o ambiguas.

Como parte de la configuración de carga del modelo, podemos destacar que se utilizó un preprocesado de la entrada donde se añaden etiquetas de prompt para que el modelo infiera mejor el mensaje y retorne su respuesta como un tensor de torch, este modelo en particular ofrecía la característica de pensamiento profundo, pero fue deshabilitada por cuestiones de rendimiento, además se requería un respuesta compacta de la forma `ID:CLASE` y cuando se ejecutaba con el modo de pensamiento, en muchas ocasiones no respetaba esta regla. Por parte de la generación se acordó un máximo de 64 tokens, ya que las respuestas estructuradas no requieren mucha cantidad de tokens, elevar este número no tenía impacto en la precisión y realentizaba la respuesta. `temperature` y `top_p` se configuraron con 0,1 ambos, debido a que el parámetro `temperature` define la creatividad de la respuesta y en nuestro caso al ser una respuesta estructurada no era conveniente, de la misma forma `top_p` define que tanto se debe usar los datos (en nuestro caso palabras) para elaborar la respuesta, anteriormente definimos salidas numéricas, por lo que las palabras no son necesarias que tengan presencia en la respuesta y como configuración de tipo de dato almacenado en el modelo, elegimos `bfloat16`, que aunque menos preciso que `float32`, es mucho más rápido su procesamiento.

8.2. Phi-4 mini

Phi-4 mini instruct se evalúa como segundo LLM para comparar el comportamiento de otro modelo instruccional ante la misma tarea. Al igual que con SmolLM3, se mantiene el mismo esquema de clases, el mismo formato de salida y el mismo criterio de evaluación. Esta consistencia es importante porque, si los prompts o ejemplos cambian entre modelos, la comparación deja de medir únicamente diferencias entre modelos.

La configuración de este modelo es exactamente igual a la del modelo SmolLM3, manteniendo tipo de dato, temperatura de generación, etc. El único cambio es que este utilizar un tokenizador al momento de crear las entradas que pasaran a la fase de generación.

8.3. Definición del Prompt

Durante las pruebas se compararon variantes de prompt y se eligió la que produjo mejor adherencia al formato solicitado y mejores resultados preliminares.

El prompt se compone de dos partes, el contexto y los textos. El objetivo del contexto es proporcionarle la información y las instrucciones específicas que debe seguir a fin de realizar la clasificación de la mejor manera posible. El contexto usado es:

Eres un clasificador de textos financieros.

Clasifica cada texto con las siguientes clases:

0 = simple

1 = complejo

Responde únicamente una línea por texto, en este formato exacto:

ID:CLASE

Ejemplo de respuesta válida:

1:0

Donde 1 es el ID y 0 es la CLASE.

Por otra parte, los textos son el conjunto de documentos que debe clasificar. Este apartado comienza con:

Textos a clasificar:

Seguido se otorgan los documentos con el formato:

Número. “Documento”

donde Número es el número de documento en ese prompt y Documento el documento (texto) a clasificar. El formato de respuesta esperado debe ser estricto para facilitar la evaluación automática:

ID:CLASE

Por ejemplo, 1:0 indica que el texto con identificador 1 fue clasificado como simple. En la implementación, cualquier línea que no cumpla el patrón exacto ID:CLASE, donde CLASE debe ser 0 o 1, se registra como -1. Esta decisión evita detener la ejecución por respuestas mal formadas y, al mismo tiempo, impide que una salida inválida sea contada como correcta. Así, el resultado refleja tanto la capacidad de clasificar como la capacidad de seguir instrucciones.

En la implementación se utiliza una clase Batch, para partir el conjunto de datos por lotes y poder procesarlo más fácilmente por los LLMs en lugar de ir texto por texto.

8.4. Clasificación sin few-shot

En la configuración sin few-shot, el prompt describe la tarea y el formato de salida, pero no proporciona ejemplos etiquetados. Esta condición evalúa la capacidad del modelo para interpretar la instrucción desde su conocimiento previo.

8.4.1. Ejecución de ejemplo

En esta sección se prueban los LLM con 3 ejemplos extraídos de FEINA para comprobar que su comportamiento es el esperado, previo a su ejecución sobre el dataset completo.

Documentos sometidos a prueba		
ID	Contenido	Clasificación
0	El varias veces mencionado Yager, nos dice acertadamente sobre estos aspectos lo siguiente: La mayoría de ustedes están hambrientos de tener libertad financiera, están hastiados de tener batallas financieras porque son como un carrusel.	1
1	El equipo realizó la redacción del libro luego de obtener la información anterior y reunir las fuentes bibliográficas y virtuales respectivas.	0
2	La mayoría de los casos involucra la implementación de estrategias de inversión muy especializadas desde el inicio.	0

Tabla 11: Esta tabla contiene los tres documentos seleccionados para ser clasificados, con su respectivo ID y clase.

Ejemplos de clasificación sin few-shot			
Documento	Etiqueta real	Predicción	
		SmolLM3	Phi-4 mini
0	1	1	0
1	0	0	1
2	0	1	1

Tabla 12: La tabla presenta las respuestas generadas por ambos LLMs. La primera columna es el ID del documento respectivo y la segunda columna representa la clasificación esperada. La tercera y cuarta columna representan las predicciones de SmolLM3 y Phi-4 mini, respectivamente. Basado en los resultados, SmolLM3 se muestra superior al tener una precisión de 0.6 mientras que Phi-4 mini tiene una precisión de 0.

8.4.2. Ejecución con el dataset completo sin few shots

En la ejecución con los 10624 datos de FEINA, donde la mitad están etiquetados como 0 y la otra mitad como 1, nos arroja los siguientes resultados:

Resultados de corrida completa		
LLM	Precisión	Tiempo
SmolLM3	0.49136902	56m41s
Phi-4 mini	0.51009483	26m55s

Tabla 13: En esta tabla tenemos en la primera columna el nombre del modelo LLM utilizado para la prueba con el dataset completo, en la segunda columna el resultado de la precisión obtenido luego de la ejecución con el dataset completo y en la última columna el tiempo que demoró en ejecutarse. En la tabla se puede apreciar que la diferencia entre la precisión de ambos modelos no es sustancial, ambos modelos aproximan un 50 %, lo cual es un poco bajo en comparación con otros resultados como regresión logística con TF-IDF o embeddings

Estos resultados pueden parecer bajos, sin embargo se debe entender que son resultados sin fase de entrenamiento, es decir, los modelos no se habían enfrentado a esos textos anteriormente y tampoco tiene ejemplos para poder deducir un resultado correcto.

8.5. Clasificación con few-shot

El few-shot learning agrega ejemplos etiquetados dentro del prompt antes de solicitar la clasificación de nuevos textos. La hipótesis es que estos ejemplos ayudan al modelo a calibrar la definición operacional de simple y complejo en el dominio financiero. Sin embargo, la literatura reciente también advierte que los LLMs pueden tener dificultades en escenarios zero-shot y few-shot cuando la tarea requiere razonamiento complejo o conocimiento específico del dominio [13].

Para que el experimento sea comparable, los mismos ejemplos deben usarse en ambos modelos y en todas las configuraciones equivalentes. Además, los ejemplos usados como shots no deben volver a evaluarse como parte del conjunto de prueba de esa configuración.

Con el fin de implementar los few shots, se modificó el prompt añadiendo una sección de shots entre el contexto y los textos. El texto que conforma esta sección es:

Ejemplos:

Seguido se agregan los shots en el formato:

Número. “Documento”

Clasificación: Clase

donde Número es el número de shot en ese prompt, Documento es un documento de ejemplo y Clase la clasificación de documento de ejemplo.

8.5.1. Ejecución de ejemplo

En esta sección se prueban los LLM con 3 ejemplos extraídos de FEINA para comprobar que su comportamiento es el esperado, previo a su ejecución sobre el dataset completo. En esta sección se prueban tres configuraciones distintas de shots, utilizando como tales los últimos 2, 4 y 7 documentos de FEINA respectivamente.

Documentos sometidos a prueba		
ID	Contenido	Clasificación
0	El varias veces mencionado Yager, nos dice acertadamente sobre estos aspectos lo siguiente: La mayoría de ustedes están hambrientos de tener libertad financiera, están hastiados de tener batallas financieras porque son como un carrusel.	1
1	El equipo realizó la redacción del libro luego de obtener la información anterior y reunir las fuentes bibliográficas y virtuales respectivas.	0
2	La mayoría de los casos involucra la implementación de estrategias de inversión muy especializadas desde el inicio.	0

Tabla 14: Esta tabla contiene los tres documentos seleccionados para ser clasificados, con su respectivo ID y clase.

Ejemplos de clasificación con few-shot							
Documento	Etiqueta real	2 Shots		4 Shots		7 Shots	
		SmolLM3	Phi-4 mini	SmolLM3	Phi-4 mini	SmolLM3	Phi-4 mini
0	1	1	0	1	1	1	0
1	0	0	0	0	0	0	0
2	0	1	1	1	1	1	1

Tabla 15: La tabla presenta las respuestas generadas por ambos LLMs. La primera columna es el ID del documento respectivo y la segunda columna representa la clasificación esperada. La tercera y cuarta columna representan las predicciones de SmolLM3 y Phi-4 mini, respectivamente. Los resultados muestran que SmolLM3 no parece verse afectado por la cantidad de shots proporcionados; incluso, no se observan diferencias entre utilizar shots y no utilizarlos, tabla 12. SmolLM3 obtuvo una precisión de $0.\bar{6}$. En el caso de Phi-4 mini, sí se observa una mejora respecto a lo mostrado sin few shots (tabla 12), obteniendo el mejor resultado con 4 shots, con una precisión del $0.\bar{6}$ en comparación con el $0.\bar{3}$ de 2 y 7 shots.

8.5.2. Ejecución con el dataset completo

En la ejecución con los 10624 datos de FEINA, donde la mitad están etiquetados como 0 y la otra mitad como 1, nos arroja los siguientes resultados:

Resultados de corrida completa				
LLM	2 Shots	Precisión		tiempo
		4 Shots	7 Shots	
SmolLM3	0.49706518381217174	0.4851085310449268	0.48815086782376504	1h25m
Phi-4 mini	0.49785539215686275	0.4958639705882353	0.508962770032174	1h23m

Tabla 16: La tabla presenta la precisión obtenida tras ejecutar ambos LLMs con 2,4 y 7 shots. Los resultados obtenidos muestran que SmolLM3 se comporta mejor con 2 shots y Phi-4 mini obtiene la mayor precisión con 7 shots. Adicionalmente, se observa que ambos modelos con 4 shots experimentan una leve disminución en su precisión. La columna de tiempo muestra el tiempo total de la corrida con todas las configuraciones de shots.

Según la tabla 16 podemos notar un resultado interesante y es que no mejora sustancialmente respecto a una configuración sin shots, con el dataset completo, de hecho podemos observar que la corrida con 2 shots logró un mejor resultado que otras configuraciones en SmolLM3 y la segunda mejor con Phi-4 mini.

9. Prueba completa

La prueba completa integra todos los tratamientos definidos anteriormente. Se realizarán 30 corridas con particiones aleatorias 80 % entrenamiento y 20 % prueba. En cada corrida, la partición debe mantenerse constante para todos los tratamientos, de modo que las diferencias observadas se atribuyan al modelo y no a variaciones en los datos evaluados.

Para registrar los resultados de forma suficiente para la prueba estadística, cada corrida debe conservar al menos el identificador de la semilla, el tratamiento evaluado, el MAE y la exactitud obtenida. Si se desea reproducir exactamente las particiones con LLMs, también conviene conservar los índices usados como prueba en cada semilla.

Los tratamientos a comparar son:

1. Regresión logística con TF-IDF.
2. Regresión logística con vectores empotrados.
3. SmolLM3 sin few-shot y con few-shot.
4. Phi-4 mini sin few-shot y con few-shot.

Resultados para la prueba completa de 30 corridas

Tratamiento	Precisión promedio	Desv. estándar Precisión
TF-IDF + regresión logística	0.4804	0.0022
Embeddings + regresión logística	0.4790	0.0030
SmolLM3	0.4976	0.0141
Phi-4 mini	0.4976	0.0141
SmolLM3 + 2 Shots	0.4983	0.0142
Phi-4 mini + 2 Shots	0.4899	0.0240

Tabla 17: En esta tabla se registran los resultados de todos los clasificadores implementados anteriormente, en la columna de tratamiento podemos encontrar la configuración del clasificador, en la columna Precisión promedio se registra la precisión media de las 30 corridas con el dataset completo y en la columna Desv. estándar Precisión la desviación estándar asociada a las 30 corridas. Se registra únicamente la configuración con 2 shots debido a que se pudo observar que con 2 shots se consiguen buenos resultados y mejora el rendimiento y la relación tiempo/precisión. Se puede observar como la diferencia entre la precisión promedio es bastante similar entre cada tratamiento utilizado en el dataset completo, mostrando que realmente ningún tratamiento es notablemente superior para el dataset completo.

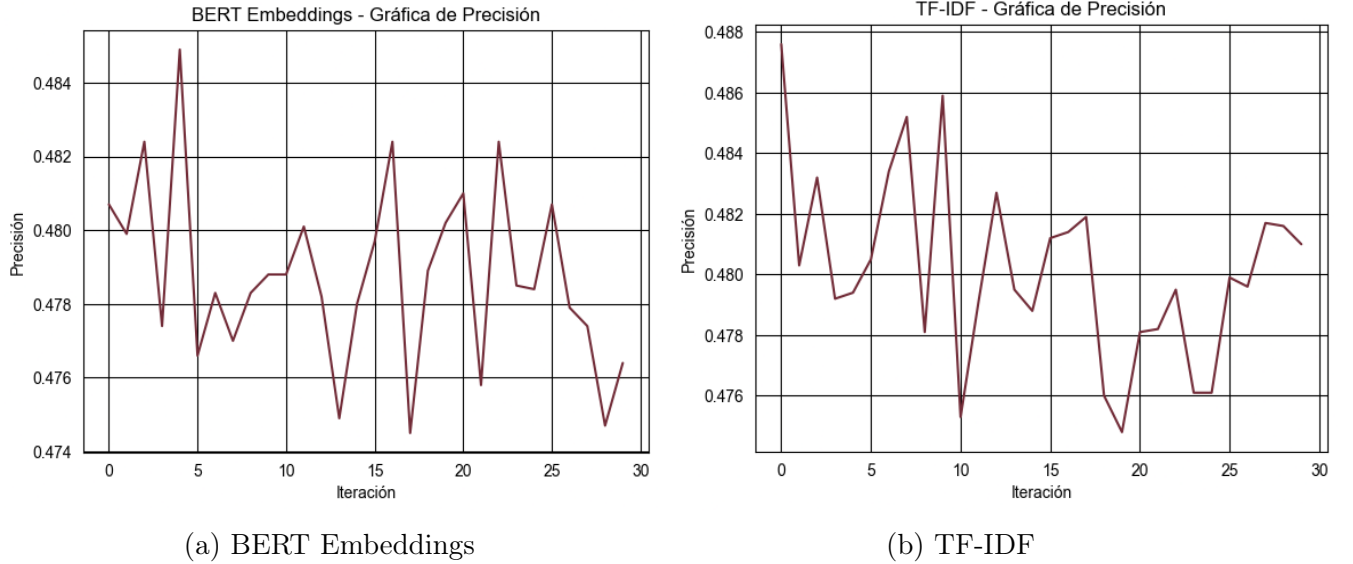
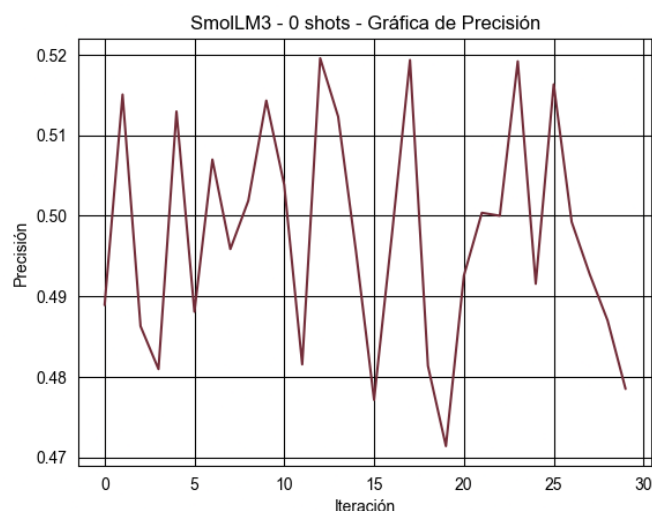
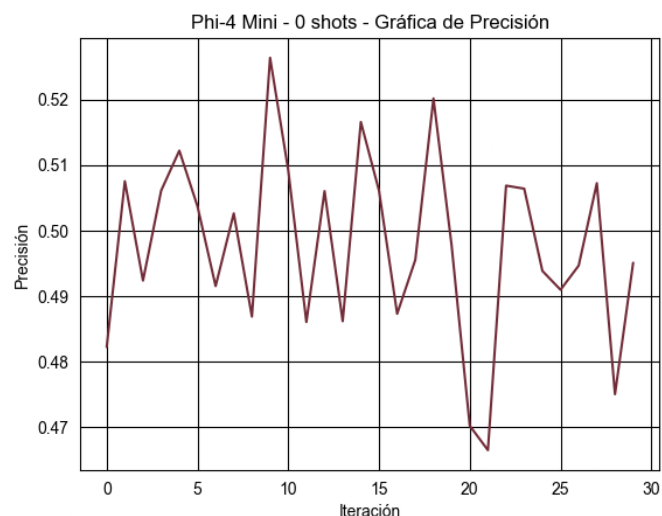


Figura 8: Gráficas de la precisión de los modelos de clasificación por cada corrida de las 30 en total. En la parte izquierda se presenta el modelo de BERT y en la derecha TF-IDF. Ambos tienen un rango de precisión y comportamiento similar. TF-IDF alcanza un máximo mayor y BERT tiene un mínimo menor.

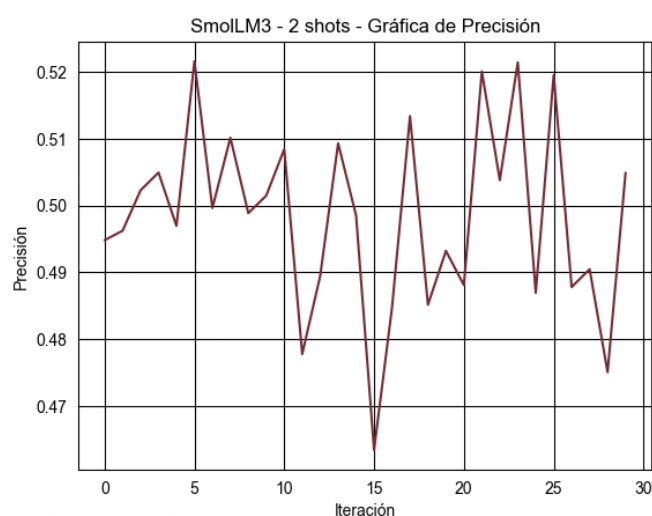


(a) SmolLM3

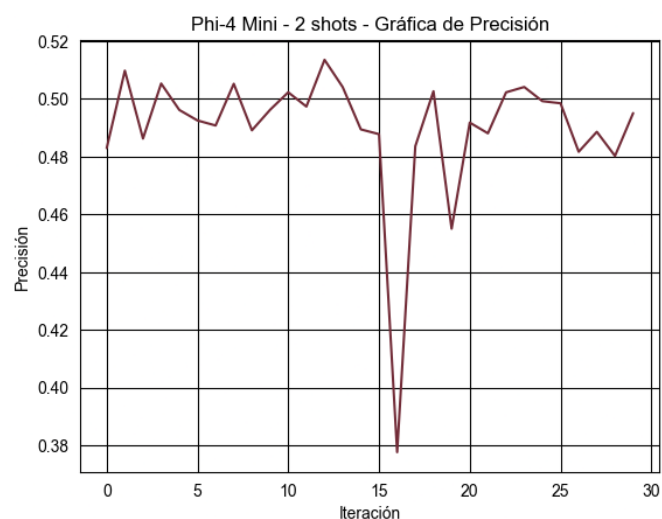


(b) Phi-4 mini

Figura 9: Gráficas de la precisión de los modelos LLM sin few-shots por cada corrida de las 30 en total. En la parte izquierda se presenta el modelo SmolLM3 y en la derecha Phi-4 mini. Ambos tienen un rango de precisión bastante similar, sin embargo, Phi-4 parece tener algo más de estabilidad pero alcanza un mínimo menor y un máximo mayor a los de SmolLM3.



(a) SmolLM3



(b) Phi4-mini

Figura 10: Gráficas de la precisión de los modelos LLM con una configuración de few-shots de 2 por cada corrida de las 30 en total. En la parte izquierda se presenta el modelo SmolLM3 y en la derecha Phi-4 mini. Phi-4 muestra algo de estabilidad en el rango de precisión $[0.48, 0.52]$ pero alcanza un mínimo considerablemente más bajo que los demás modelos, esto explica por qué es el que tiene mayor desviación estándar de todos. Por otro lado, SmolLM3 muestra buena precisión durante las iteraciones comparado con otros.

En la tabla 17 podemos derivar que aunque los tratamientos de TF-IDF y embeddings

mostraban mejores resultados en el dataset completo con una corrida, luego de las 30 corridas con los datos aleatorios, hizo que su precisión bajara considerablemente al punto que establecer una diferencia clara entre cuál tratamiento usar para tener mejores resultados es una tarea difícil. Otros datos que caben resaltar son que el modelo con mayor precisión media fue SmolLM3, mientras que mantuvo la menor desviación estándar media fue el tratamiento de regresión logística con TF-IDF.

10. Prueba estadística

Después de obtener las 30 corridas por tratamiento, se aplicó una prueba estadística para determinar si las diferencias observadas entre los modelos eran sistemáticas o si podían explicarse por variaciones aleatorias entre corridas. En este experimento no basta con comparar únicamente los promedios, porque cada tratamiento fue evaluado varias veces y cada corrida puede favorecer ligeramente a un modelo distinto según la partición de los datos.

El diseño de comparación es pareado: la corrida i de TF-IDF, BERT, SmolLM3 y Phi-4 corresponde al mismo número de corrida dentro del protocolo experimental. Por tanto, las observaciones no se tratan como grupos completamente independientes, sino como mediciones comparables corrida por corrida.

Los tratamientos evaluados fueron:

- TF-IDF con regresión logística.
- BERT con regresión logística.
- SmolLM3 sin shots.
- SmolLM3 con 2 shots.
- Phi-4 mini sin shots.
- Phi-4 mini con 2 shots.

Corrida / seed	TF-IDF	BERT	SmolLM3 sin shots	SmolLM3 + 2 shots	Phi-4 sin shots	Phi-4 + 2 shots
1	0.4876	0.4807	0.4889	0.4948	0.4822	0.4830
2	0.4803	0.4799	0.5151	0.4962	0.5075	0.5098
3	0.4832	0.4824	0.4863	0.5023	0.4924	0.4863
4	0.4792	0.4774	0.4809	0.5050	0.5061	0.5053
5	0.4794	0.4849	0.5130	0.4970	0.5122	0.4962
6	0.4805	0.4766	0.4881	0.5216	0.5037	0.4925
7	0.4834	0.4783	0.5070	0.4996	0.4915	0.4908
8	0.4852	0.4770	0.4959	0.5102	0.5026	0.5052
9	0.4781	0.4783	0.5019	0.4989	0.4869	0.4891
10	0.4859	0.4788	0.5143	0.5015	0.5264	0.4962
11	0.4753	0.4788	0.5038	0.5084	0.5092	0.5023
12	0.4791	0.4801	0.4815	0.4778	0.4861	0.4974
13	0.4827	0.4782	0.5196	0.4895	0.5060	0.5136
14	0.4795	0.4749	0.5123	0.5093	0.4862	0.5041
15	0.4788	0.4780	0.4955	0.4985	0.5166	0.4895
16	0.4812	0.4797	0.4771	0.4634	0.5061	0.4878
17	0.4814	0.4824	0.4978	0.4847	0.4873	0.3777
18	0.4819	0.4745	0.5193	0.5134	0.4955	0.4836
19	0.4760	0.4789	0.4813	0.4851	0.5202	0.5027
20	0.4748	0.4802	0.4714	0.4932	0.4977	0.4551
21	0.4781	0.4810	0.4926	0.4881	0.4702	0.4918
22	0.4782	0.4758	0.5004	0.5201	0.4665	0.4881
23	0.4795	0.4824	0.5000	0.5038	0.5069	0.5023
24	0.4761	0.4785	0.5192	0.5214	0.5064	0.5041
25	0.4761	0.4784	0.4916	0.4869	0.4939	0.4992
26	0.4799	0.4807	0.5163	0.5195	0.4910	0.4985
27	0.4796	0.4779	0.4992	0.4878	0.4947	0.4817
28	0.4817	0.4774	0.4928	0.4905	0.5072	0.4886
29	0.4816	0.4747	0.4870	0.4751	0.4751	0.4803
30	0.4810	0.4764	0.4785	0.5049	0.4951	0.4951

Tabla 18: Valores de precisión usados en la prueba estadística. Cada fila representa una corrida, corresponde también al valor del seed en la corrida y cada columna un tratamiento.

Como se comparan más de dos tratamientos sobre las mismas 30 corridas, se utilizó la prueba de Friedman mediante la función `friedmanchisquare` de `scipy.stats`. Esta prueba es no paramétrica, por lo que no exige asumir que los valores de precisión siguen una distribución normal. En lugar de trabajar directamente con los valores absolutos, compara la posición relativa de los tratamientos dentro de cada corrida.

La idea central de Friedman es la siguiente: en cada corrida, los tratamientos se ordenan según su rendimiento. Como la métrica usada es la precisión, un valor mayor indica mejor desempeño. El mejor tratamiento de una corrida recibe el mejor rango, el siguiente recibe el segundo mejor rango, y así sucesivamente. Si todos los tratamientos fueran equivalentes, sus rangos promedio deberían ser parecidos. Si un tratamiento tiende a quedar sistemáticamente mejor o peor posicionado, sus rangos se separan de los demás [14].

La hipótesis nula se define como:

$$H_0 : \text{todos los tratamientos presentan un rendimiento equivalente.} \quad (13)$$

La hipótesis alternativa se define como:

$$H_1 : \text{al menos un tratamiento presenta un rendimiento sistemáticamente distinto.} \quad (14)$$

El estadístico observado de Friedman se denota como χ_F^2 y se calcula a partir de los rangos acumulados de cada tratamiento:

$$\chi_F^2 = \frac{12}{nk(k+1)} \sum_{j=1}^k R_j^2 - 3n(k+1), \quad (15)$$

donde n es la cantidad de corridas, k es la cantidad de tratamientos y R_j es la suma de rangos del tratamiento j . En este experimento se tiene:

$$n = 30, \quad k = 6. \quad (16)$$

El estadístico χ_F^2 mide qué tan diferentes son los rangos acumulados entre tratamientos. Si los tratamientos se comportaran de forma similar, las sumas de rangos serían parecidas y el estadístico sería pequeño. Si algunos tratamientos quedan repetidamente por encima o por debajo de otros, el estadístico aumenta.

Una vez obtenido el valor observado χ_F^2 , se calcula el valor p . Este valor representa la probabilidad de obtener una diferencia entre rangos igual o más extrema que la observada, suponiendo que la hipótesis nula fuera cierta. Para Friedman, esta probabilidad se aproxima usando una distribución chi-cuadrado con $k - 1$ grados de libertad:

$$p = P(\chi_{k-1}^2 \geq \chi_F^2). \quad (17)$$

Como en este caso hay 6 tratamientos, se utilizan:

$$k - 1 = 5 \quad (18)$$

grados de libertad. Si el valor p es menor que el nivel de significancia elegido, usualmente $\alpha = 0,05$, se rechaza la hipótesis nula.

Tratamiento	Accuracy promedio	Desv. estándar	Rango medio
TF-IDF + regresión logística	0.480177	0.003060	4.933
BERT + regresión logística	0.478773	0.002447	5.333
SmolLM3 sin shots	0.497615	0.014070	2.467
SmolLM3 + 2 shots	0.498288	0.014151	2.533
Phi-4 mini sin shots	0.497646	0.014093	2.567
Phi-4 mini + 2 shots	0.489933	0.024004	3.167

Tabla 19: Resumen de los tratamientos evaluados en la prueba estadística. Como la métrica usada es precisión, un rango medio menor representa una mejor posición relativa dentro de las corridas.

La prueba de Friedman produjo el siguiente resultado:

$$\chi_F^2 = 72,230181, \quad p = 3,5174 \times 10^{-14}. \quad (19)$$

Como $p < 0,05$, se rechaza la hipótesis nula. Esto indica que no todos los tratamientos presentan el mismo rendimiento. Dicho de otra forma, las diferencias observadas entre los modelos no parecen deberse únicamente al azar de las corridas.

Además, se puede calcular el coeficiente de Kendall W_K como tamaño de efecto global de la prueba de Friedman:

$$W_K = \frac{\chi_F^2}{n(k-1)}. \quad (20)$$

Sustituyendo los valores del experimento:

$$W_K = \frac{72,230181}{30(6-1)} = 0,481535. \quad (21)$$

Este valor mide qué tan consistente es el orden relativo de los tratamientos entre corridas. Un valor cercano a 0 indicaría poco acuerdo entre corridas, mientras que un valor cercano a 1 indicaría que los tratamientos mantienen casi siempre el mismo orden. En este caso, $W_K = 0,481535$ sugiere un efecto global apreciable: existe una tendencia clara, aunque todavía hay variabilidad entre corridas.

Como Friedman solo indica que existe alguna diferencia global, pero no dice entre cuáles tratamientos ocurre, se aplicó una prueba post-hoc de Wilcoxon pareada para cada par de tratamientos. Esta prueba compara dos tratamientos a la vez usando las diferencias corrida por corrida sin asumir normalidad en las diferencias. Su estadístico se denota como W y se calcula a partir de los rangos de las diferencias absolutas entre ambos tratamientos [15].

En esta parte, W no debe confundirse con el coeficiente de Kendall W_K . El W de Wilcoxon pertenece a una comparación por pares, mientras que W_K resume el tamaño de efecto global de Friedman.

Como hay muchas comparaciones por pares, aumenta el riesgo de encontrar una diferencia significativa por casualidad. Para controlar ese problema, se aplicó la corrección de Holm usando `multiptest` de `statsmodels.stats.multitest`. Esta corrección ajusta los valores p obtenidos en las comparaciones post-hoc. Por eso, la decisión final se toma con p_{Holm} y no con el valor p crudo.

Comparación	W	p_{Holm}	Diferencia significativa
TF-IDF vs BERT	152.000	0.513638	No
TF-IDF vs SmolLM3 sin shots	17.000	$4,62681 \times 10^{-6}$	Sí
TF-IDF vs SmolLM3 + 2 shots	19.000	$6,29015 \times 10^{-6}$	Sí
TF-IDF vs Phi-4 mini sin shots	21.000	$8,32602 \times 10^{-6}$	Sí
TF-IDF vs Phi-4 mini + 2 shots	55.000	0.000792205	Sí
BERT vs SmolLM3 sin shots	12.000	$1,95578 \times 10^{-6}$	Sí
BERT vs SmolLM3 + 2 shots	15.000	$3,57255 \times 10^{-6}$	Sí
BERT vs Phi-4 mini sin shots	15.000	$3,57255 \times 10^{-6}$	Sí
BERT vs Phi-4 mini + 2 shots	53.000	0.000711959	Sí
SmolLM3 sin shots vs SmolLM3 + 2 shots	231.500	1.000000	No
SmolLM3 sin shots vs Phi-4 mini sin shots	231.000	1.000000	No
SmolLM3 sin shots vs Phi-4 mini + 2 shots	138.000	0.513638	No
SmolLM3 + 2 shots vs Phi-4 mini sin shots	208.000	1.000000	No
SmolLM3 + 2 shots vs Phi-4 mini + 2 shots	140.000	0.403816	No
Phi-4 mini sin shots vs Phi-4 mini + 2 shots	151.000	0.601753	No

Tabla 20: Comparaciones post-hoc mediante Wilcoxon pareado con corrección de Holm. Se considera diferencia significativa cuando $p_{\text{Holm}} < 0,05$.

Los resultados muestran que TF-IDF y BERT no presentan una diferencia estadísticamente significativa entre sí, ya que su valor corregido es $p_{\text{Holm}} = 0,513638$. Esto significa que, aunque sus promedios no son exactamente iguales, la diferencia no es suficientemente consistente en las 30 corridas como para afirmar que uno supera claramente al otro.

En cambio, tanto TF-IDF como BERT sí presentan diferencias significativas frente a las configuraciones basadas en LLMs. Esto se observa en los valores $p_{\text{Holm}} < 0,05$ para las comparaciones contra SmolLM3 y Phi-4 mini.

Por otro lado, no se encontraron diferencias significativas entre SmolLM3, Phi-4 mini y sus variantes con 2 shots. Esto sugiere que, bajo este protocolo experimental, agregar 2 ejemplos como few-shot no produjo una mejora estadísticamente distinguible dentro de los LLMs evaluados.

En síntesis, la prueba de Friedman indica que existen diferencias globales entre los tratamientos. El análisis post-hoc muestra que esas diferencias se concentran principalmente entre los métodos basados en regresión logística y los modelos generativos. Sin embargo, dentro de las variantes de LLMs no hay evidencia estadística suficiente para declarar un ganador claro. Además, aunque algunas diferencias sean estadísticamente significativas, los valores de precisión siguen estando cerca del azar, por lo que la significancia estadística no debe confundirse con un rendimiento práctico alto para la tarea de clasificación de complejidad textual.

11. Conclusiones

Este trabajo compara representaciones clásicas, representaciones semánticas y modelos generativos instruidos para la clasificación de complejidad textual en español financiero. La estructura experimental permite observar no solo qué modelo obtiene mejores métricas, sino también qué tipo de representación resulta más adecuada para un dominio especializado y sensible a decisiones lingüísticas como tokenización, eliminación de palabras vacías y lematización.

Como conclusión, no podemos determinar un mejor modelo para la tarea, ya que todos los modelos utilizan métodos diferentes para intentar clasificar los datos. Pero podemos analizar

los resultados obtenidos, por ejemplo, para regresión logística con vectores empotrados si se notó una precisión bastante más alta que otros tratamientos, aunque al final la brecha fue disminuyendo. Por parte del TF-IDF aunque la idea de clasificar textos entre simples y complejos utilizando la frecuencia de los términos es buena, en la práctica no resultó de la mejor forma, porque aunque en la prueba con el dataset completo (Ver tabla 7) en las 30 corridas también bajó considerablemente su precisión. Tampoco podemos discriminar el uso de LLMs, ya que siendo justos, no pasa por un proceso de entrenamiento controlado por nosotros mismos para el problema de clasificación, únicamente utiliza su preentrenamiento para intentar resolver tareas de otros ámbitos y esto es una clara desventaja, además el costo computacional y tiempo que se demora estos tratamientos, es demasiado y sus resultados no son favorables. Lo mismo pasa con TD-IDF, aunque claramente es menos costoso que LLMs, sigue siendo costoso en términos de tiempo, por lo que podemos decir que el mejor tratamiento en relación a la precisión que ofrece respecto a su tiempo de ejecución es regresión logística con vectores empotrados, aunque no por decir que concluimos que fue el mejor desde el punto de vista de tiempo, signifique que cumplió bien su trabajo, ya que una precisión del 50 % roza el azar y no podemos considerar un buen clasificador de textos complejos/simples con estas precisiones.

Referencias

- [1] N. Perez-Rojas, S. Calderon-Ramirez, M. Solis-Salazar, M. Romero-Sandoval, M. Arias-Monge, and H. Saggion, “A novel spanish dataset for financial education text simplification targeting visually impaired individuals,” *IEEE Access*, vol. 13, pp. 87 472–87 484, 2025.
- [2] F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958.
- [3] S. Calderón, “Clasificación binaria,” https://tecnube1-my.sharepoint.com/personal/sacalderon_itcr_ac_cr/_layouts/15/onedrive.aspx?viewid=cfe0f8c2%2Dbae%2D4a0c%2D95bb%2Dec07593bbb11&id=%2Fpersonal%2Fsacalderon%5Fitcr%5Fac%5Fcr%2FDocuments%2FTEC%2FInteligencia%5FArtificial%5FI%5FSemestre%5F2026%2FPublico%2FApuntos%2FClasificacion%2Epdf&parent=%2Fpersonal%2Fsacalderon%5Fitcr%5Fac%5Fcr%2FDocuments%2FTEC%2FInteligencia%5FArtificial%5FI%5FSemestre%5F2026%2FPublico%2FApuntos, 2023.
- [4] C. Murphy, P. Gray, and G. Stewart, “Verified perceptron convergence theorem,” in *Proceedings of the 1st ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*. Association for Computing Machinery, 2017. [Online]. Available: <https://doi.org/10.1145/3088525.3088673>
- [5] D. R. Cox, “The regression analysis of binary sequences,” *Journal of the Royal Statistical Society. Series B (Methodological)*, vol. 20, no. 2, pp. 215–232, 1958.
- [6] M. P. LaValley, “Logistic regression,” *Circulation*, vol. 117, no. 18, pp. 2395–2399, 2008.

- [7] G. Raskutti, M. J. Wainwright, and B. Yu, “Early stopping and non-parametric regression: An optimal data-dependent stopping rule,” *arXiv preprint arXiv:1306.3574*, 2013. [Online]. Available: <https://doi.org/10.48550/arXiv.1306.3574>
- [8] H. Choi, D. Choi, and H. Lee, “Early stopping based on unlabeled samples in text classification,” in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, S. Muresan, P. Nakov, and A. Villavicencio, Eds. Dublin, Ireland: Association for Computational Linguistics, May 2022, pp. 708–718. [Online]. Available: <https://aclanthology.org/2022.acl-long.52/>
- [9] S. Calderón, “Introducción al procesamiento del lenguaje natural,” https://tecnubel-my.sharepoint.com/:b:/r/personal/sacalderon_iter_ac_cr/Documents/TEC/Inteligencia_Artificial_I_Semestre_2026/Publico/Apuntes/NLP_intro.pdf?csf=1&web=1&e=3FTpny, 2022.
- [10] M. Schmidt, “Convergence of gradient descent,” <https://www.cs.ubc.ca/~schmidtm/Courses/540-W18/L4.pdf>, 2017, lecture notes, University of British Columbia.
- [11] M. Umer, Z. Imtiaz, M. Ahmad, M. Nappi, C. Medaglia, G. S. Choi, and A. Mehmood, “Impact of convolutional neural network and fasttext embedding on text classification,” *Multimedia Tools and Applications*, vol. 82, no. 4, pp. 5569–5585, 2023. [Online]. Available: <https://doi.org/10.1007/s11042-022-13459-x>
- [12] G. Jawahar, B. Sagot, and D. Seddah, “What does BERT learn about the structure of language?” in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, A. Korhonen, D. Traum, and L. Màrquez, Eds. Florence, Italy: Association for Computational Linguistics, Jul. 2019, pp. 3651–3657. [Online]. Available: <https://aclanthology.org/P19-1356/>
- [13] I. Chamieh, T. Zesch, and K. Giebertmann, “LLMs in short answer scoring: Limitations and promise of zero-shot and few-shot approaches,” in *Proceedings of the 19th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2024)*. Mexico City, Mexico: Association for Computational Linguistics, 2024, pp. 309–315. [Online]. Available: <https://aclanthology.org/2024.bea-1.25/>
- [14] J. T. E. Richardson, “Friedman test,” in *The SAGE Encyclopedia of Educational Research, Measurement, and Evaluation*, B. B. Frey, Ed. SAGE Publications, 2018, pp. 709–712. [Online]. Available: <https://doi.org/10.4135/9781506326139.n277>
- [15] J. D. Gibbons, “Wilcoxon test,” in *The SAGE Encyclopedia of Social Science Research Methods*, M. S. Lewis-Beck, A. Bryman, and T. F. Liao, Eds. Thousand Oaks, CA: SAGE Publications, Inc., 2004. [Online]. Available: <https://doi.org/10.4135/9781412950589.n1087>